

LABORATORIO 14 · CAPSTONE

¿Cómo se demuestra que está cerrada?

*TLS, SASL y ACL. El único laboratorio del curso que se aprueba fallando.
Guía para hacerlo por tu cuenta, paso a paso*

Confluent Platform 8.2.0 · 3 brokers KRaft con SASL_SSL · PKI propia

Duración estimada · 50 minutos

NovaTech Logistics · caso de estudio

Antes de empezar

Qué vas a lograr

Éste es el último laboratorio del curso, y **el único que se aprueba fallando**. De los comandos del recorrido, dos **tienen que dar error**. Si alguno de los dos funcionara, el clúster estaría roto.

Y los dos errores **no se parecen en nada**. Esa diferencia es lo único que este laboratorio tiene que dejarte.

LO QUE ESTE LABORATORIO DEMUESTRA

La seguridad se demuestra con lo que rechaza, no con lo que permite.

Y ÉSTA ES LA DISTINCIÓN, DICHA DE FRENTE ANTES DE EMPEZAR

No la deduzcas del contraste. **Léela ahora, y vuelve a leerla al terminar.**

Lo que le pasa al cliente	Qué falló
Un cliente sin credenciales que ni siquiera se conecta . No hay conversación, no hay respuesta, no hay mensaje de error del clúster	Autenticación
Un cliente con credenciales válidas que entra al clúster y aun así no puede leer el tópico . El clúster le contesta, y le dice por qué, nombrando el recurso	Autorización

Autenticar es saber quién eres. Autorizar es saber qué puedes tocar. Son dos compuertas distintas, en ese orden, y hacen falta las dos. Si te vas sin entender esa diferencia, el laboratorio no sirvió.

Qué necesitas tener listo

Requisito	Cómo lo compruebas
Docker Desktop corriendo, con 6 GB de RAM	Ajustes de Docker Desktop, pestaña <i>Resources</i>
Git Bash abierto en la carpeta del lab	<pre>cd Capitulo_5/lab-14-capstone-resiliencia-seguridad</pre>
Los puertos 9092, 9093, 9094 y 8090 libres	El paso 1 te avisa si no lo están
Paciencia en el arranque	El paso 1 tarda más que los demás labs porque genera criptografía. Medido, 33 segundos

TODA LA CRIPTOGRAFÍA YA ESTÁ HECHA CUANDO LLEGAS

No vas a generar un certificado, ni armar un almacén de claves, ni editar un *listener*. **Eso ocupaba la mitad del recorrido viejo y está en el *para profundizar A***, porque no es lo que hay que aprender aquí. Lo que hay que aprender es a leer dos errores.

El caso

La conversación sobre seguridad en un clúster suele terminar en una demostración que no demuestra nada.

Alguien conecta un cliente, produce un mensaje, lo consume, y dice *listo, está seguro*. Y todos asienten, porque funcionó.

Pero que funcione no prueba nada sobre la seguridad. Un clúster completamente abierto también deja producir y consumir. La demostración es idéntica en los dos casos. **Lo único que distingue a un clúster seguro de uno abierto es lo que el seguro no deja hacer**, y eso no se ve haciendo las cosas que sí funcionan.

Y hay una segunda mitad, que es donde de verdad se pierde la gente. Cuando por fin alguien hace la prueba negativa, la hace **una sola vez**. Intenta conectar sin credenciales, ve que falla, y da el tema por cerrado.

Eso deja fuera la mitad del problema, y es la mitad cara. Porque una cosa es **quién eres** y otra muy distinta es **qué puedes tocar**. En SUNAT esa diferencia tiene nombre y apellido. El sistema de un área tiene credenciales perfectamente válidas, porque es un sistema de la casa y tiene que poder entrar, **y aun así no debe poder leer los datos de otra área**.

Un clúster que confunde esas dos cosas deja entrar a todos los que tienen llave. Y una vez dentro, **todos pueden todo**.

LA METÁFORA · EL ÚLTIMO DÍA, Y SON DOS PIEZAS

Kafka sigue siendo **el restaurante**. El **mozo** es el broker, el **tipo de comanda** es el tópico, los **sectores del salón** son las particiones, las **libretas de respaldo** son las réplicas y la **brigada** es el grupo de consumidores.

Las piezas de hoy son dos, y la gracia está en no confundirlas.

En el restaurante	En Kafka
La puerta de servicio con cerradura. Si no tienes llave no entras, y desde fuera ni siquiera oyes lo que pasa dentro	La autenticación , o sea SASL sobre TLS
La lista pegada en la cocina que dice qué puede tocar cada uno de los que ya entraron	La autorización , o sea las ACL

Y ahora el ejemplo que hay que tener en la cabeza todo el laboratorio.

Hay un proveedor que **tiene llave** de la puerta de servicio. Entra todos los días, deja la mercadería en el depósito, y se va. Es de la casa, la llave es suya y está bien que la tenga.

Y no puede entrar a la caja. No porque le falte la llave de la puerta, que ésa la tiene. Es porque la lista de la cocina no lo nombra para eso.

Tener llave y tener permiso son dos cosas distintas, y hacen falta las dos.

Las tres piezas, con sus nombres

CONCEPTO · TLS

Cifra la conversación entre el cliente y el broker, y le permite al cliente comprobar que el broker es quien dice ser. **Es el sobre cerrado**. Sin él, un usuario y una contraseña viajarían a la vista de cualquiera.

CONCEPTO · SASL

El mecanismo con el que el cliente se presenta. Aquí es **PLAIN**, o sea usuario y contraseña. **Se llama PLAIN y no significa que vaya en claro**, porque va dentro del TLS. El nombre confunde a todo el mundo la primera vez.

CONCEPTO · ACL

Access Control List. Las reglas de qué **principal** puede hacer qué **operación** sobre qué **recurso**. Se guardan **en el propio clúster**, no en un archivo de configuración, y se consultan en cada petición.

CONCEPTO · PRINCIPAL

El nombre con el que el clúster conoce a quien se conecta. Aquí son `User:app1` y `User:app2`, y salen del usuario con el que se autenticaron.

Los tres personajes

Quién	Qué puede	Su papel en el laboratorio
<code>admin</code>	Super user. Todo permitido, sin ACL de por medio	El que prueba que el tópico confidencial sí se puede leer
<code>app1</code>	Escribe y lee los dos tópicos	El que siembra los mensajes
<code>app2</code>	Solo lee el público	El protagonista. Tiene llave buena y le van a decir que no

Levantar el clúster seguro

Qué vamos a hacer

Encender el laboratorio. **Este arranque hace mucho más que los anteriores**, y por eso tarda.

El comando

```
cd Capitulo_5/lab-14-capstone-resiliencia-seguridad
chmod +x bin/*.sh kafka-cli/*.sh
bin/start-lab.sh
```

bin/start-lab.sh — genera **la PKI completa**, o sea la autoridad certificadora y los certificados firmados de los tres brokers. Arma los almacenes de claves y de confianza, levanta los tres brokers con TLS y SASL, crea los dos tópicos y escribe las ACL. **Medido, 33 segundos**

EL FINAL DE LO QUE VAS A VER · RECORTADO

CLÚSTER NOVATECH LAB 12 OPERATIVO (con seguridad)

Componentes:

Broker 1 SASL_SSL: localhost:9092
Broker 2 SASL_SSL: localhost:9093
Broker 3 SASL_SSL: localhost:9094
Kafbat UI: http://localhost:8090

Tópicos del Lab 14:

novatech.lab12.publico - 3 part, RF=3 (autenticados con ACL)
novatech.lab12.confidencial - 3 part, RF=3 (solo admin + app1)

Usuarios:

admin → super user, todo permitido
app1 → producer+consumer publico Y confidencial
app2 → SOLO consumer del público

DICE «LAB 12» Y ESTÁS EN EL 14

No es un error. Los tópicos se llaman `novatech.lab12.*` y el banner conserva ese número porque el material viene de una numeración anterior. **Es el laboratorio 14**, y todo lo demás calza.

VAS BIEN SI...

Los tres brokers dicen `SASL_SSL` y no `PLAINTEXT`. Ésa es la diferencia con los trece laboratorios anteriores, y es toda la diferencia.

Compruébalo también en Docker:

```
docker ps --filter "label=com.docker.compose.project=novatech-lab14" \
  --format 'table {{.Names}}\t{{.Status}}'
```

SI EL ARRANQUE SE QUEDA MUCHO TIEMPO GENERANDO CERTIFICADOS

Es normal que sea el arranque más lento del curso. Medido, 33 segundos en total. Si pasa de dos minutos, córtalo con `Ctrl+C` y vuelve a lanzarlo. Los certificados se regeneran desde cero y no queda nada a medias.

SI ALGÚN BROKER NO LLEGA A *HEALTHY*

Con TLS los brokers arrancan más lento que en los otros labs, porque tienen que cargar los almacenes de claves. Dale otro minuto.

Si sigue, mira si el problema son los certificados:

```
docker logs --tail 40 kafka-broker-1 | grep -i -E "ssl|keystore|truststore"
```

Sembrar los mensajes

Qué vamos a hacer

Escribir un mensaje en cada tópico, como `appl`, que es el que tiene permiso para los dos.

ESTE PASO NO ES OPCIONAL, Y ES FÁCIL SALTÁRSELO

El `bin/start-lab.sh` crea los dos tópicos **vacíos** y no escribe nada en ellos. Compruébalo si quieres, antes de sembrar.

Sin este paso, el laboratorio no se puede demostrar. El paso 5 daría cero mensajes por falta de permiso y el paso 6 daría cero mensajes porque el tópico está vacío, **y las dos cosas se ven exactamente igual**. No podrías distinguir «me lo denegaron» de «no había nada», que es justo lo que este laboratorio existe para distinguir.

Los comandos

```
kafka-cli/produce-publico.sh "Comprobante publico de la clase"
kafka-cli/produce-confidencial.sh "Comprobante confidencial de la clase"
```

produce-publico.sh — escribe un mensaje en `novatech.lab12.publico`, autenticándose como `appl`

produce-confidencial.sh — lo mismo en `novatech.lab12.confidencial`. **Funciona porque `appl` tiene `WRITE` sobre los dos tópicos**, y lo vas a ver escrito en el paso 3

el texto entre comillas — el mensaje. **Elige algo reconocible**, porque lo vas a buscar en la salida de los pasos 5 y 6

LO QUE VAS A VER, EN CADA UNO

```
Mensaje: Comprobante publico de la clase
Warning: --producer.config is deprecated and will be removed in a future version. Use --
command-config instead.
✓ Mensaje publicado
```

EL AVISO DE `DEPRECATED` SALE SIEMPRE

Aparece en **todos** los comandos de este laboratorio, en los que funcionan y en los que no. Es el envoltorio usando un flag viejo que Kafka 8 todavía acepta. **No es tuyo y no afecta a nada.**

VAS BIEN SI...

Los dos dijeron `✓ Mensaje publicado` . Y hay un mensaje en cada tópico:

```
docker exec cli-client bash -c 'kafka-get-offsets \
  --bootstrap-server kafka-broker-1:9092 \
  --command-config /etc/kafka/client-properties/admin.properties \
  --topic novatech.lab12.publico --time -1'
```

```
novatech.lab12.publico:0:0
novatech.lab12.publico:1:0
novatech.lab12.publico:2:1
```

Una de las tres particiones tiene 1 y las otras 0. Cuál te toque a ti depende del reparto, y da igual. Lo que importa es que la suma sea 1.

SI EL PRODUCTOR FALLA CON ALGO SOBRE SSL O HANDSHAKE

El clúster todavía no terminó de arrancar. **Vuelve al paso 1** y espera a que los tres brokers estén *healthy*. Con TLS eso tarda más que en los labs anteriores.

SI DICE `TOPICAUTHORIZATIONEXCEPTION` AL PRODUCIR

Las ACL no se escribieron al arrancar. Míralas con el comando del paso 3. **Si la lista sale vacía**, vuelve a levantar el laboratorio con `bin/start-lab.sh` .

Las reglas, escritas

Qué vamos a hacer

Leer las reglas antes de probar nada. Están en el clúster y se consultan.

El comando

```
kafka-cli/list-acls.sh
```

kafka-cli/list-acls.sh — envoltorio del curso. Por dentro llama a `kafka-acls --list` con las credenciales de `admin`, porque consultar las reglas también requiere permiso

LO QUE VAS A VER

```
Current ACLs for resource `ResourcePattern(resourceType=TOPIC, name=novatech.lab12.publico, patternType=LITERAL)`:
```

```
(principal=User:app1, host=*, operation=CREATE, permissionType=ALLOW)
(principal=User:app2, host=*, operation=READ, permissionType=ALLOW)
(principal=User:app1, host=*, operation=READ, permissionType=ALLOW)
(principal=User:app2, host=*, operation=DESCRIBE, permissionType=ALLOW)
(principal=User:app1, host=*, operation=DESCRIBE, permissionType=ALLOW)
(principal=User:app1, host=*, operation=WRITE, permissionType=ALLOW)
```

```
Current ACLs for resource `ResourcePattern(resourceType=TOPIC, name=novatech.lab12.confidencial, patternType=LITERAL)`:
```

```
(principal=User:app1, host=*, operation=CREATE, permissionType=ALLOW)
(principal=User:app1, host=*, operation=READ, permissionType=ALLOW)
(principal=User:app1, host=*, operation=DESCRIBE, permissionType=ALLOW)
(principal=User:app1, host=*, operation=WRITE, permissionType=ALLOW)
```

```
Current ACLs for resource `ResourcePattern(resourceType=GROUP, name=*, patternType=LITERAL)`:
```

```
(principal=User:app2, host=*, operation=READ, permissionType=ALLOW)
(principal=User:app1, host=*, operation=READ, permissionType=ALLOW)
```

El orden de los tres bloques cambia entre corridas. Si a ti sale el de `GROUP` en el medio, es lo mismo.

Cómo se lee

Cada línea es una regla. **Quién**, desde **dónde**, **qué operación**, y si se permite o se niega.

Y ahora la parte importante, **que es lo que no está**. Mira el bloque de `novatech.lab12.confidencial` y cuenta los principales.

Recurso	Aparece <code>app1</code>	Aparece <code>app2</code>
<code>novatech.lab12.publico</code>	Sí, con 4 operaciones	Sí, con <code>READ</code> y <code>DESCRIBE</code>
<code>novatech.lab12.confidencial</code>	Sí, con 4 operaciones	No aparece. Ni una línea.

ESA AUSENCIA ES TODA LA AUTORIZACIÓN DE ESTE LABORATORIO

No hay una regla que diga «a `app2` se le niega el confidencial». Hay algo más fuerte. No hay ninguna regla sobre `app2` en ese tópico, y **en Kafka lo que no está explícitamente permitido está denegado.**

Fíjate también en el bloque de `resourceType=GROUP`. **Leer un tópico no basta.** Hay que tener permiso sobre el grupo de consumidores también, y es el permiso que más se olvida al configurar un cliente nuevo.

VAS BIEN SI...

Puedes señalar con el dedo **dónde no está `app2`**. **Anota una respuesta.** ¿Cuántas reglas hay que nieguen algo explícitamente?

SI LA LISTA SALE VACÍA O DA ERROR DE AUTORIZACIÓN

Consultar las ACL también requiere permiso, y el envoltorio usa `admin` para eso. Si falla, el clúster no terminó de arrancar o las ACL no se escribieron. **Vuelve a levantar el laboratorio.**

El que no tiene llave

Qué vamos a hacer

Primera prueba negativa. Vamos a pedirle al clúster la cosa más inocente que existe, que es la lista de tópicos, **sin presentar ninguna credencial.**

PREDICE ANTES DE EJECUTAR, Y ESCRÍBELO

¿Qué va a contestar el clúster? *¿No autorizado? ¿Usuario o contraseña incorrectos? ¿Otra cosa?*
Contesta antes de teclear.

El comando

```
kafka-cli/attempt-no-auth.sh
```

kafka-cli/attempt-no-auth.sh — corre `kafka-topics --bootstrap-server kafka-broker-1:9092 --list` dentro del contenedor `cli-client`

el flag que no está — lo que importa de este comando es lo que le falta. No lleva `--command-config`, así que el cliente se presenta sin usuario, sin contraseña y sin certificado

LO QUE VAS A VER · RECORTADO, LA TRAZA SIGUE

```
Error while executing topic command : The AdminClient thread has exited. Call: listTopics
[2026-08-29 04:34:20,327] ERROR org.apache.kafka.common.errors.TimeoutException: The
AdminClient thread has exited. Call: listTopics
(org.apache.kafka.tools.TopicCommand)
[2026-08-29 04:34:20,329] ERROR Uncaught exception in thread 'kafka-admin-client-thread |
adminclient-1': (org.apache.kafka.common.utils.KafkaThread)
java.lang.OutOfMemoryError: Java heap space
    at java.base/java.nio.HeapByteBuffer.<init>(Unknown Source)
    at java.base/java.nio.ByteBuffer.allocate(Unknown Source)
    at org.apache.kafka.common.memory.MemoryPool$1.tryAllocate(MemoryPool.java:30)
    at org.apache.kafka.common.network.NetworkReceive.readFrom(NetworkReceive.java:103)
    at org.apache.kafka.common.network.KafkaChannel.receive(KafkaChannel.java:462)
```

Cómo se lee

Esa salida asusta, y es la respuesta correcta. Vale la pena entenderla, porque es la firma exacta de este error y la vas a reconocer en producción.

Lo que dice	Qué significa de verdad
<code>OutOfMemoryError: Java heap space</code>	El cliente habló en claro contra un puerto que solo entiende TLS. Lo que le contestó el broker fue el saludo de TLS , que el cliente interpretó como si fuera un mensaje de Kafka. Leyó los primeros bytes como «longitud del mensaje», le salió un número gigantesco, e intentó reservar esa memoria
<code>TimeoutException: The AdminClient thread has exited</code>	Y como el hilo murió, la petición nunca se contestó

Y AHORA FÍJATE EN LO QUE EL CLÚSTER NO DIJO

No dijo *no autorizado*. No dijo *usuario incorrecto*. No dijo *el tópico no existe*. **No dijo nada.**

Desde fuera de la puerta no se oye nada de lo que pasa dentro. El cliente no sabe si el clúster existe, qué tópicos tiene, ni siquiera si hay alguien al otro lado. **Eso es autenticación. No es que te digan que no. Es que no hay conversación.**

VAS BIEN SI...

Salió el `OutOfMemoryError` y el `TimeoutException`, y el clúster **no nombró ningún tópico ni ningún usuario.**

Compara con tu predicción. ¿Esperabas un mensaje de «no autorizado»?

SI EL COMANDO DEVUELVE LA LISTA DE TÓPICOS

Eso sí sería un problema grave, y significaría que el *listener* no está exigiendo SASL_SSL. Comprueba en el banner del paso 1 que los tres brokers dicen `SASL_SSL`. Si dicen `PLAINTEXT`, el laboratorio no está montado como debe y hay que volver a levantarlo.

SI TE SALE OTRO ERROR EN VEZ DEL `OUTOFMEMORYERROR`

Puede pasar, porque depende de cómo caiga la temporización del *handshake*. **Lo que tiene que cumplirse es lo de fondo**, o sea que el cliente muere sin que el clúster le conteste nada útil, y que en ningún momento aparece el nombre de un tópico. Si ves un `SSLHandshakeException` o un `timeout` a secas, la lección es la misma.

El que tiene llave pero no permiso

Qué vamos a hacer

Segunda prueba negativa, y es la que este laboratorio existe para dejar.

Ahora `app2` va a intentar leer el tópico confidencial. `app2` tiene credenciales válidas, que son las que usa todos los días para su trabajo, y en el paso 6 las vas a ver funcionar.

PREDICE OTRA VEZ

¿El error se va a parecer al del paso 4?

El comando

```
kafka-cli/consume-confidencial-app2.sh
```

`--consumer.config ../app2.properties` — aquí sí hay credenciales, y son buenas. El archivo trae `security.protocol=SASL_SSL`, el usuario `app2`, su contraseña y el almacén de confianza

`--topic novatech.lab12.confidencial` — el tópico sobre el que `app2` no aparece en ninguna regla

`--timeout-ms 5000` — cuánto espera sin recibir antes de rendirse. Sin esto quedaría colgado

LO QUE VAS A VER

```
[2026-08-29 04:34:28,187] WARN [Consumer clientId=console-consumer, groupId=console-consumer-94008] The metadata response from the cluster reported a recoverable issue with correlation id 2 : {novatech.lab12.confidencial=TOPIC_AUTHORIZATION_FAILED}
(org.apache.kafka.clients.NetworkClient)
[2026-08-29 04:34:28,189] ERROR [Consumer clientId=console-consumer, groupId=console-consumer-94008] Topic authorization failed for topics [novatech.lab12.confidencial]
(org.apache.kafka.clients.Metadata)
[2026-08-29 04:34:28,190] ERROR Error processing message, terminating consumer process:
(org.apache.kafka.tools.consumer.ConsoleConsumer)
org.apache.kafka.common.errors.TopicAuthorizationException: Not authorized to access topics:
[novatech.lab12.confidencial]
Processed a total of 0 messages
```

Cómo se lee

Compara esta salida con la del paso 4, línea por línea. No se parecen en nada, y ahí está la lección entera.

	Paso 4 · sin credenciales	Paso 5 · <code>app2</code>
¿Hubo conversación?	No. El cliente murió sin respuesta	Sí. El clúster contestó
¿El clúster dijo por qué?	No dijo nada	Sí, <code>TOPIC_AUTHORIZATION_FAILED</code>
¿Nombró el recurso?	No sabía ni que existía	Sí, <code>[novatech.lab12.confidencial]</code>
Qué error salió	<code>OutOfMemoryError</code>	<code>TopicAuthorizationException</code>
Qué falló	La autenticación	La autorización

LO QUE HAY QUE DECIR EN VOZ ALTA

`app2` **entró al clúster.** Se autenticó bien, negoció TLS, pidió los metadatos, y el clúster se los negó **por nombre.**

Eso solo puede pasar si el clúster sabe quién es. **Para negarle algo a alguien, primero hay que saber quién es alguien.**

Autenticar es saber quién eres. Autorizar es saber qué puedes tocar. Son dos compuertas distintas, en ese orden, y hacen falta las dos.

VAS BIEN SI...

Salió `TOPIC_AUTHORIZATION_FAILED`, con el nombre del tópico entre corchetes, y `Processed a total of 0 messages`.

Anota una respuesta. ¿Cómo sabes, mirando solo este error, que `app2` sí consiguió conectarse al clúster?

SI SALE UN `OUTOFMEMORYERROR` COMO EL DEL PASO 4

Entonces las credenciales no se están usando. Comprueba que el archivo existe:

```
docker exec cli-client cat /etc/kafka/client-properties/app2.properties
```

Si no lo encuentra, el laboratorio no está bien montado y hay que volver a levantarlo.

SI APP2 SÍ CONSIGUE LEER EL CONFIDENCIAL

Alguien le agregó una ACL, y lo más probable es que hayas hecho el bloque B del *para profundizar* y no la hayas borrado. **Bórrala**, con el comando del `--remove` que está allí, y vuelve a probar.

La misma llave, otra puerta

Qué vamos a hacer

Si el laboratorio terminara en el paso 5 quedaría una duda razonable. *¿Y cómo sé que las credenciales de `app2` sirven para algo? A lo mejor están mal.*

Es una buena objeción, y se contesta con **el mismo comando, cambiando solo el tópico.**

El comando

```
kafka-cli/consume-publico.sh
```

LO QUE VAS A VER

```
Comprobante publico de la clase
[2026-08-29 04:35:49,473] ERROR Error processing message, terminating consumer process:
(org.apache.kafka.tools.consumer.ConsoleConsumer)
org.apache.kafka.common.errors.TimeoutException
Processed a total of 1 messages
```

UNA TRAMPA DE LECTURA QUE HAY QUE ATAJAR

Esta salida también trae la palabra `ERROR`, y sin embargo funcionó.

Ese `TimeoutException` del final no es un fallo. Es el `--timeout-ms 5000` cumpliéndose porque, después de leer el mensaje, no llegó nada más. **Sale en todos los comandos de este laboratorio, en los que funcionan y en los que no.**

La regla de lectura de este laboratorio. No mires la palabra `ERROR`. Mira **cuál** es el error, y mira **la última línea.**

Última línea	Qué pasó
<code>Processed a total of 0 messages</code>	No leyó nada
<code>Processed a total of 1 messages</code>	Leyó

La comparación que cierra el argumento

Comando	Usuario	Tópico	Última línea
Paso 5	app2	confidencial	Processed a total of 0 messages
Paso 6	app2	publico	Processed a total of 1 messages

MISMAS CREDENCIALES, MISMO COMANDO, DISTINTO TÓPICO, DISTINTO RESULTADO

Eso prueba que lo que falló en el paso 5 no era la llave. Era el permiso.

Y para cerrar del todo, el que sí puede leerlo

```
kafka-cli/consume-confidencial-admin.sh
```

LO QUE VAS A VER

```
Comprobante confidencial de la clase
[2026-08-29 04:35:59,071] ERROR Error processing message, terminating consumer process:
(org.apache.kafka.tools.consumer.ConsoleConsumer)
org.apache.kafka.common.errors.TimeoutException
Processed a total of 1 messages
```

El mismo tópico que le negaron a `app2`, leído sin problema por `admin`.

Y ADMIN NO APARECE EN NINGUNA ACL

Vuelve a mirar la salida del paso 3 si no me crees. `admin` es **super user**, una llave maestra configurada en el broker, **a la que las reglas ni se le consultan**. Ése es el motivo de que en producción el *super user* deba ser de muy pocas manos, y de que nadie deba usarlo para el día a día. **Es el único principal del clúster para el que la lista de la cocina no existe.**

LO LOGRASTE SI...

Tienes las tres últimas líneas anotadas y son **0**, **1** y **1**. Y puedes explicar por qué el primer cero no significa lo mismo que un tópico vacío.

Y la pregunta que de verdad importa. Cuéntale a alguien, en dos frases, la diferencia entre lo que pasó en el paso 4 y lo que pasó en el paso 5. Si puedes hacerlo sin mirar la guía, el curso cumplió.

SI EL PASO 6 TAMBIÉN TE DA 0 MESSAGES

El tópico está vacío, y te saltaste el paso 2. Es el error más importante de este laboratorio, porque hace que los pasos 5 y 6 se vean idénticos y la demostración se cae. Vuelve al paso 2 y siembra los mensajes.

Lo que aprendiste

1 Una demostración que solo muestra lo que funciona no demuestra nada.

Un clúster abierto da exactamente la misma demostración que uno seguro. Pide siempre la prueba negativa. **Y pide las dos**, o sea sin credenciales, y con credenciales válidas sobre algo que no le toca. La segunda casi nunca se hace, y es la que protege los datos entre áreas.

2 Los dos fallos se distinguen mirando el error, y son cosas distintas.

Un cliente que **muere sin respuesta** es autenticación, y quiere decir que ni siquiera entró. Un `TopicAuthorizationException` que **nombra el recurso** es autorización, y quiere decir que el cliente **sí entró** y el clúster sabe quién es. Para negarle algo a alguien, primero hay que saber quién es.

3 En Kafka, lo que no está explícitamente permitido está denegado.

La protección del tópico confidencial no es una regla que niega. Es **la ausencia** de una regla que permita. Cuando revises la seguridad de un clúster, no busques las prohibiciones. Cuenta los permisos y pregunta por los que sobran.

4 El super user no tiene ACL, y por eso es el mayor riesgo del clúster.

Debe ser de pocas manos, no debe usarse para operación diaria, y su contraseña merece el mismo cuidado que la de un administrador de base de datos. De los sistemas que hoy se conectan a tus clústeres, ¿cuántos usan credenciales de super user porque era más fácil?

Para profundizar

A · La PKI, que el arranque hizo por ti

Generar la autoridad certificadora, firmar los certificados de los tres brokers, armar los almacenes y configurar los *listeners*. Todo eso ocupaba la mitad del recorrido viejo y lo hace un script.

```
cat bin/generate-certs.sh
cat infra/client-properties/app2.properties
```

Lo que hay que mirar en `app2.properties`, que son cinco líneas y explican todo el paso 5.

```
security.protocol=SASL_SSL
sasl.mechanism=PLAIN
sasl.jaas.config=org.apache.kafka.common.security.plain.PlainLoginModule required
username="app2" password="app2-secret";
ssl.truststore.location=/etc/kafka/secrets/kafka.truststore.jks
ssl.truststore.password=changeit
```

PLAIN NO SIGNIFICA QUE LA CONTRASEÑA VIAJE EN CLARO

Significa que el **mecanismo de presentación** es usuario y contraseña, y va **dentro del túnel TLS** que establece `security.protocol=SASL_SSL`. Si dijera `SASL_PLAINTEXT`, entonces sí iría a la vista.

La pregunta que vale. ¿Dónde está guardada la contraseña de `app2` en este laboratorio, y por qué eso no sería aceptable en producción?

B · Escribir una ACL nueva, y verla surtir efecto

La forma de comprobar que entendiste el paso 5 es arreglarlo, y después volver a romperlo.

```
MSYS_NO_PATHCONV=1 docker exec cli-client kafka-acls \
  --bootstrap-server kafka-broker-1:9092 \
  --command-config /etc/kafka/client-properties/admin.properties \
  --add --allow-principal User:app2 \
  --operation READ --operation DESCRIBE \
  --topic novatech.lab12.confidencial

kafka-cli/consume-confidencial-app2.sh
```

MSYS_NO_PATHCONV=1 — solo para Git Bash en Windows. Sin esto, Git Bash ve el `/etc/kafka/...` de la línea siguiente y lo traduce a una ruta de Windows antes de que Docker la vea. Fuera de Windows nadie

lee esta variable

LO QUE SALE DEL `--ADD`

```
Adding ACLs for resource `ResourcePattern(resourceType=TOPIC, name=novatech.lab12.confidencial,
patternType=LITERAL)` :
    (principal=User:app2, host=*, operation=DESCRIBE, permissionType=ALLOW)
    (principal=User:app2, host=*, operation=READ, permissionType=ALLOW)
```

Y DEL CONSUMIDOR, QUE EN EL PASO 5 DABA CERO

```
Comprobante confidencial de la clase
Processed a total of 1 messages
```

Lo que hay que mirar. El comando que fallaba en el paso 5 ahora funciona, **sin reiniciar nada y sin tocar un archivo**. Las ACL viven en el clúster y se consultan en cada petición.

Y AQUÍ EL ENVOLTORIO TE VA A MENTIR

Va a seguir imprimiendo `[Resultado esperado] error tipo 'Not authorized'` y Eso PRUEBA que las ACLs funcionan: `app2 NO puede leer confidencial`. **Es un texto fijo del script, escrito para el caso del paso 5, y ahora está diciendo lo contrario de lo que pasó.** Mira la última línea, que es la que cuenta.

Y después bórrala, o el laboratorio deja de demostrar lo que dice.

```
MSYS_NO_PATHCONV=1 docker exec cli-client kafka-acls \
  --bootstrap-server kafka-broker-1:9092 \
  --command-config /etc/kafka/client-properties/admin.properties \
  --remove --allow-principal User:app2 \
  --operation READ --operation DESCRIBE \
  --topic novatech.lab12.confidencial --force
```

El `--remove` con `--force` **no imprime nada**, y eso es lo correcto, porque el `--force` es justamente lo que salta la pregunta de confirmación. Compruébalo volviendo a correr el consumidor, que tiene que dar cero otra vez.

ESE IDA Y VUELTA ES LO QUE CONVIERTE ESTO EN UNA DEMOSTRACIÓN

Medido, **cero, uno, cero**. Una compuerta que solo sabe decir que no es una pared. Una que solo sabe decir que sí no es una compuerta.

C · El drill de *failover*

La otra mitad del capstone. Qué pasa con los datos cuando se cae un broker.

```
kafka-cli/describe-confidencial.sh
kafka-cli/produce-confidencial.sh "pedido-pre-fallo-1"
kafka-cli/simulate-failure.sh 3
kafka-cli/describe-confidencial.sh
kafka-cli/produce-confidencial.sh "pedido-durante-fallo-1"
kafka-cli/recover-broker.sh 3
kafka-cli/describe-confidencial.sh
kafka-cli/verify-no-loss.sh
```

Lo que hay que mirar. El `describe` antes, durante y después. Con factor de réplica 3 y `min.insync.replicas=2`, el clúster **sigue aceptando escrituras con un broker caído**, y al volver, el broker se pone al día solo. El `verify-no-loss.sh` comprueba que no se perdió ningún mensaje.

La pregunta que vale. ¿Qué pasaría con `min.insync.replicas=3` sobre factor de réplica 3 cuando se cae un broker? *Y compárala con lo que viste en el laboratorio 05.*

D · El capstone automatizado

```
bin/run-capstone.sh
```

Corre el laboratorio entero de punta a punta y deja un informe. **Es la forma de comprobar el clúster después de tocarlo**, y lo que usarías tú si mañana cambias una ACL en producción.

E · Los recorridos viejos, completos

Las seis guías originales quedaron en `_fuente-extra/guia/`. TLS y certificados, SASL, ACL, `min.insync.replicas`, la simulación de fallo y el capstone. **Ahí está todo lo que este recorrido comprimió**, con sus actividades y sus preguntas.

F · El reporte del laboratorio

`plantillas/reporte-entregable.md`, con las respuestas de referencia en `soluciones/reporte-resuelto.md`.

Antes de cerrar

DEJA EL TERRENO LIMPIO

```
bin/90-test-lab.sh    # ver el estado del laboratorio
bin/stop-lab.sh      # apagar los contenedores
```

Si hiciste el bloque B del *para profundizar*, comprueba antes que borraste la ACL. Si no, el laboratorio queda sin demostrar lo que dice, y la próxima vez que lo levantes el paso 5 te va a salir mal.

LO QUE TE LLEVAS DE LOS CATORCE

Si hay que resumirlo en una línea, **Kafka no te avisa.**

No te avisa cuando borra por retención. No te avisa cuando un contrato cambia. No te avisa cuando un consumidor se queda atrás. No te avisa cuando alguien lee lo que no le toca, ni cuando alguien lo intenta.

Todo lo que aprendiste en catorce laboratorios es a preguntar. Y hoy aprendiste la pregunta más difícil de todas, que es la que se hace sobre lo que no pasó. **Tener llave y tener permiso son dos cosas distintas, y hacen falta las dos.**